实验一 基础电路的实现与测试

一、实验目的

1. 掌握 Verilog 语言框架，编程及调试的方法。
2. 熟悉 Verilog 的基本语法。
3. 掌握 Vivado 开发平台及 FPGA 开发板的使用。

二、实验内容

1. 利用“Vivado Test”目录下的实例，在 Vivado 中完成一个工程的设计、编辑、综合和实现的全过程。
2. 完成基本电路模块的设计与测试（只执行 SIMULATION 与 RTL ANALYSIS，不用下载到开发板），说明电路功能，分析设计代码实现、RTL 电路图、仿真代码及仿真结果。

三、实验要求

1. 说明电路功能，分析设计、仿真代码和电路图。
2. 分析仿真波形，观察输入输出是否与预期电路功能相符（测试要全面，关注特殊情况的测试）。
3. 记录设计和调试过程。

四、实验过程及结果分析

4.1 Vivado 开发流程验证

拍照记录开发板运行结果，简要分析对开发过程和三种文件作用的理解

1、文件分析：

（1）设计文件（Design）：构建逻辑基础，实现预期功能。在本例中实现 3 位输入到 8 位输出的译码基本逻辑。

（2）约束文件（Constraints）：联系硬件，构建输入输出。在本例中将逻辑信号绑定到 BASYS3 的对应物理引脚，实现 LED 灯的对应功能。

（3）仿真文件（Simulation）：在烧录硬件前进行仿真，验证设计是否正确。在本例中自行设计初值，验证译码器的输入输出逻辑，避免硬件调试错误。

2、开发过程：

（1）创建项目，编写代码文件。

（2）进行仿真，验证电路功能正确。（波形图、电路图）

（3）Synthesis：将 HDL 代码转换为门级网表，对资源利用率和性能进行优化。

（4）Implementation：把网表映射到具体的 FPGA 资源上。

（5）生成 bitstream：用于烧录硬件。

（6）将代码烧录到硬件，并进行调试。

3、代码分析：

（1）Design：

```
module Decoder38(
    input a,
    input b,
    input c,
    output reg [7:0] y,
    output reg [10:0] display_out);

    always@(*) begin
        case({a,b,c})
            3'b000 : begin y = 8'b0000_0001; display_out <= 11'b1011_0000001;end
            3'b001 : begin y = 8'b0000_0010; display_out <= 11'b1011_1001111;end
            3'b010 : begin y = 8'b0000_0100; display_out <= 11'b1011_0010010;end
            3'b011 : begin y = 8'b0000_1000; display_out <= 11'b1011_0000110;end
            3'b100 : begin y = 8'b0001_0000; display_out <= 11'b1011_1001100;end
            3'b101 : begin y = 8'b0010_0000; display_out <= 11'b1011_0100100;end
            3'b110 : begin y = 8'b0100_0000; display_out <= 11'b1011_0100000;end
            3'b111 : begin y = 8'b1000_0000; display_out <= 11'b1011_0001111;end
            default : begin y = 'b1111_1111; display_out <= 11'b1111_1111111;end
        endcase
    end

endmodule
```

定义一个模块 Decoder38，接收 3 位二进制输入信号 a, b, c，输出 8 位译码信号 y 以及 11 位的 display_out 用于数码管显示。使用 case 语句判断输入的三位值，根据不同的输入激活不同的输出位，对应的 y 为输出，display_out 则用来控制数码管显示，模拟七段码。

（2）Constraints：

```
## Switches
set_property PACKAGE_PIN W16 [get_ports a]
        set_property IOSTANDARD LVCMOS33 [get_ports a]
set_property PACKAGE_PIN V16 [get_ports b]
        set_property IOSTANDARD LVCMOS33 [get_ports b]
set_property PACKAGE_PIN V17 [get_ports c]
```

```
        set_property IOSTANDARD LVCMOS33 [get_ports c]

## LEDs
set_property PACKAGE_PIN V14 [get_ports {y[7]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[7]}]
set_property PACKAGE_PIN U14 [get_ports {y[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[6]}]
set_property PACKAGE_PIN U15 [get_ports {y[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[5]}]
set_property PACKAGE_PIN W18 [get_ports {y[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[4]}]
set_property PACKAGE_PIN V19 [get_ports {y[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[3]}]
set_property PACKAGE_PIN U19 [get_ports {y[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[2]}]
set_property PACKAGE_PIN E19 [get_ports {y[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[1]}]
set_property PACKAGE_PIN U16 [get_ports {y[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {y[0]}]

##7 segment display
set_property PACKAGE_PIN W4 [get_ports {display_out[10]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[10]}]
set_property PACKAGE_PIN V4 [get_ports {display_out[9]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[9]}]
set_property PACKAGE_PIN U4 [get_ports {display_out[8]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[8]}]
set_property PACKAGE_PIN U2 [get_ports {display_out[7]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[7]}]
set_property PACKAGE_PIN W7 [get_ports {display_out[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[6]}]
set_property PACKAGE_PIN W6 [get_ports {display_out[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[5]}]
set_property PACKAGE_PIN U8 [get_ports {display_out[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[4]}]
set_property PACKAGE_PIN V8 [get_ports {display_out[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[3]}]
set_property PACKAGE_PIN U5 [get_ports {display_out[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[1]}]
set_property PACKAGE_PIN V5 [get_ports {display_out[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[2]}]
set_property PACKAGE_PIN U7 [get_ports {display_out[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {display_out[0]}]
```

设置输入输出接口，将 display_out 与硬件对应引脚相连，使 LED 在接收到相应输入时

能够符合预期进行工作。

（3）Simulation：

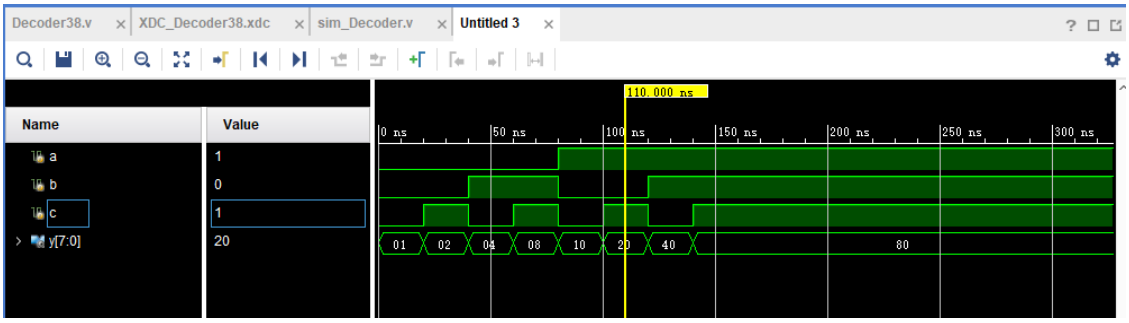
```
module sim_Decoder();
    reg a, b, c;
    wire[7:0] y;

    Decoder38 test(a,b,c,y);

    initial begin
        {a,b,c} = 3'b000; #20;
        {a,b,c} = 3'b001; #20;
        {a,b,c} = 3'b010; #20;
        {a,b,c} = 3'b011; #20;
        {a,b,c} = 3'b100; #20;
        {a,b,c} = 3'b101; #20;
        {a,b,c} = 3'b110; #20;
        {a,b,c} = 3'b111; #20;
    end
endmodule
```

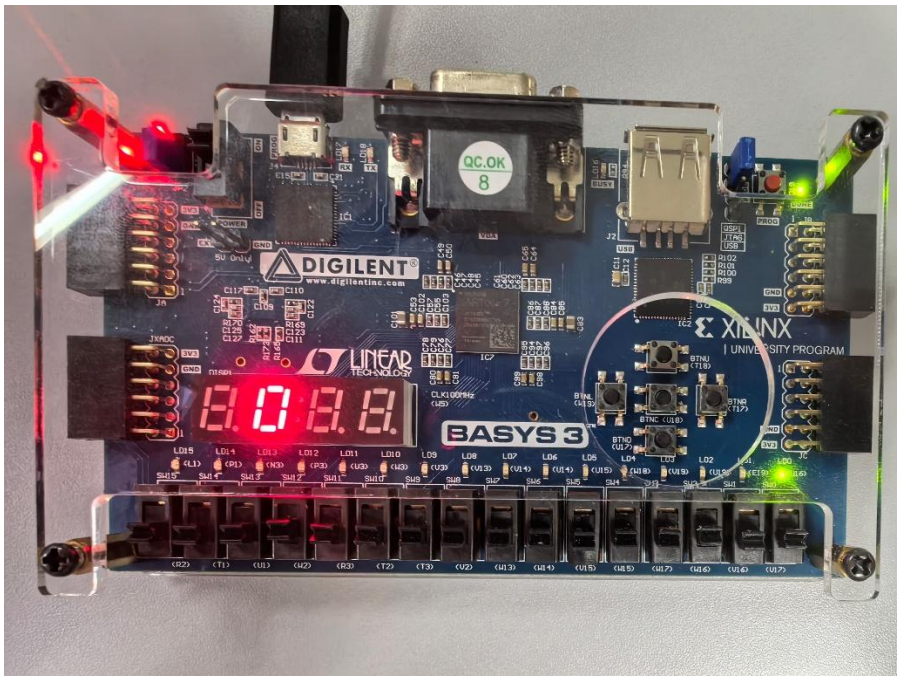
设置输入 abc 的八种情况，每隔 20 时间单位更新一次，以便测试所有情况。通过波形图观察 y 的输出情况。

4、仿真波形图：

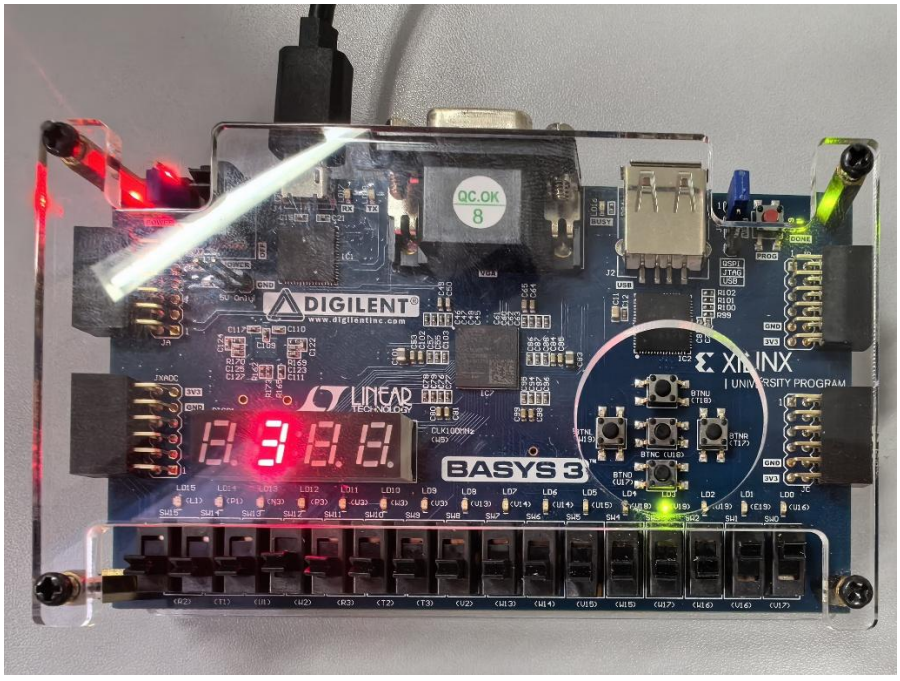


将输入 abc 转换为八位二进制数，进行译码，abc 对应的十进制数位置为 1，如 101→5→0010000，第五位为 1，因此输出十六进制数为 20。

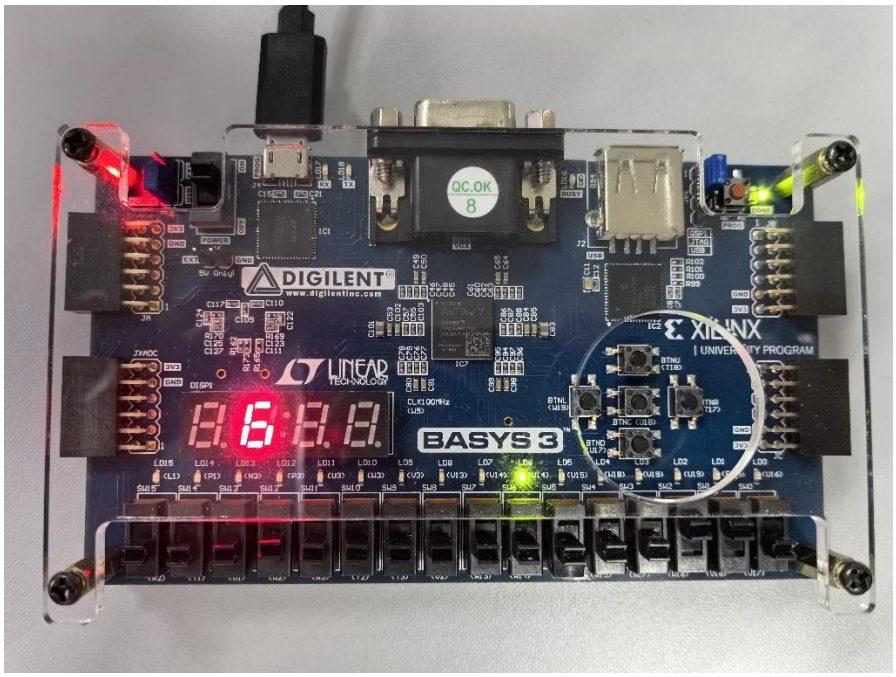
5、BASYS3 烧录完成后照片：



在无输入时，0 号 LED 亮，显示 0。



SW0、SW1 输入时，为 011，3 号 LED 亮，显示 3。



SW2、SW1 输入时，为 110，6 号 LED 亮，显示 6。
测试表明，译码器能够正常工作，符合代码的预期。

4.2 基础电路的实现与测试

对电路功能、设计代码、电路图、测试结果等进行简要分析说明，不能只贴截图。（代码请贴文字，而不是图片。电路图和仿真波形图等最好用电脑截图的图片，而不是手机拍照的图片。）

4.2.1 and21 的电路设计

（1）代码分析：

Design：

```
module and21(D0, D1, Y);
    input D0;
    input D1;
    output Y;
    wire D0, D1, Y;

    assign Y = D0 & D1;
endmodule
```

输入 D0，D1，进行与操作后输出 Y。

Simulation：

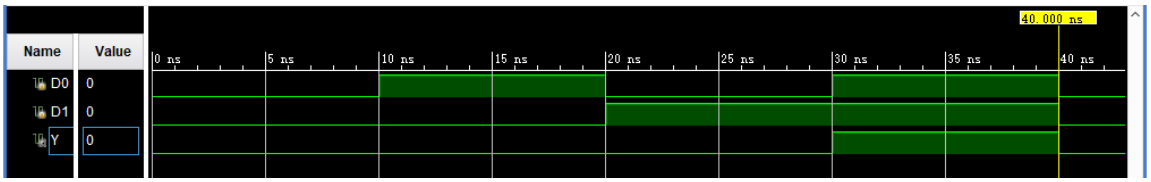
```
module sim_and21;
    reg D0, D1;
    wire Y;

    initial begin
        D0 = 0; D1= 0;
        #10 D0 = 1; D1 = 0;
        #10 D0 = 0; D1 = 1;
        #10 D0 = 1; D1 = 1;
        #10 D0 = 0; D1 = 0;
    end

    and21 and21t(D0, D1, Y);
endmodule
```

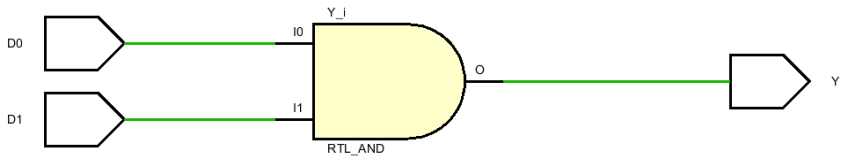
设置 D0、D1 值的四种情况，每 10 时间单位更新一次，观察输出 Y。

（2）SIMULATION：



从波形中可看出，只有 D0、D1 均为 1 时 Y 输出 1，其余情况皆为 0，符合与门特征。

（3）RTL ANALYSIS：



4.2.2 and61 的电路设计

（1）代码分析：

Design：

```
module and61(
    input D0, D1, D2, D3, D4, D5,
    output Y);

    wire Y1, Y2, Y3, Y4;
```

```
    and21 and21_1(D0, D1, Y1),
          and21_2(D2, D3, Y2),
          and21_3(D4, D5, Y3),
          and21_4(Y1, Y2, Y4),
          and21_5(.D0(Y3), .Y(Y), .D1(Y4));
endmodule
```

通过实例化多个 and21 子模块构建多层与逻辑结构，将 6 个输入两两组合求与，逐层合并。最终 Y 仅在 6 个输入全为 1 时输出 1。结构上采用模块重用、分层设计思想，提升可维护性。

Simulation：

```
module sim_and61();
    reg D0, D1, D2, D3, D4, D5;
    wire Y;
    integer i; //循环变量
    initial begin
        //两种赋值方法
        D0 = 0; D1 = 0; D2 = 0; D3 = 0; D4 = 0; D5 = 0; //初始化赋值
        #5 {D0, D1, D2, D3, D4, D5} = 6'b100000;        //位拼接赋值
        fork    //fork/join 语句，并行执行
            forever #5 D1 = ~D1;    //一直执行，每隔 5 单位翻转一次
            repeat(5) #10 D2 = ~D2; //重复 5 次，每隔 10 单位翻转一次
        join
    end

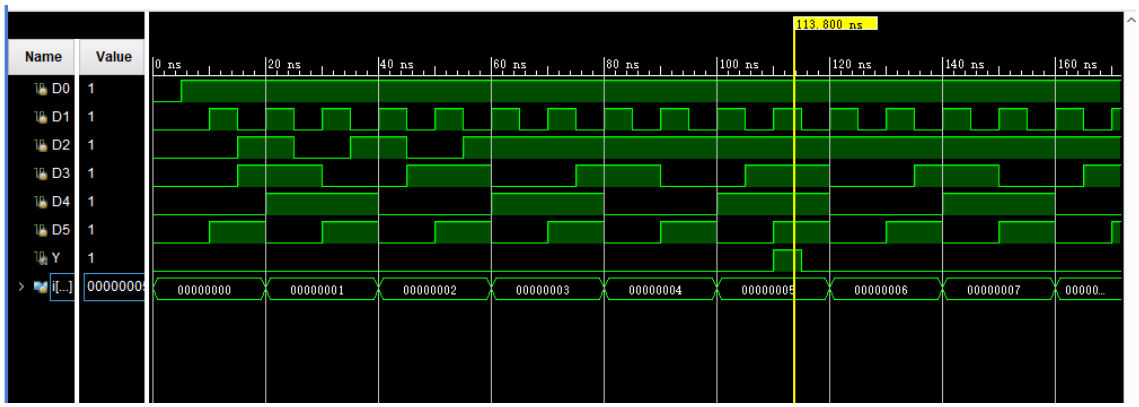
    initial fork
        while(1) begin #15 D3 = ~D3; end
        for (i = 0; i < 10; i = i + 1) #20 D4= ~D4; //两种循环
    join

    always begin
        #10 D5 = ~D5;   //每隔 10 时间单位翻转，一直执行
    end

    and61 and61_1(D0, D1, D2, D3, D4, D5, Y);
endmodule
```

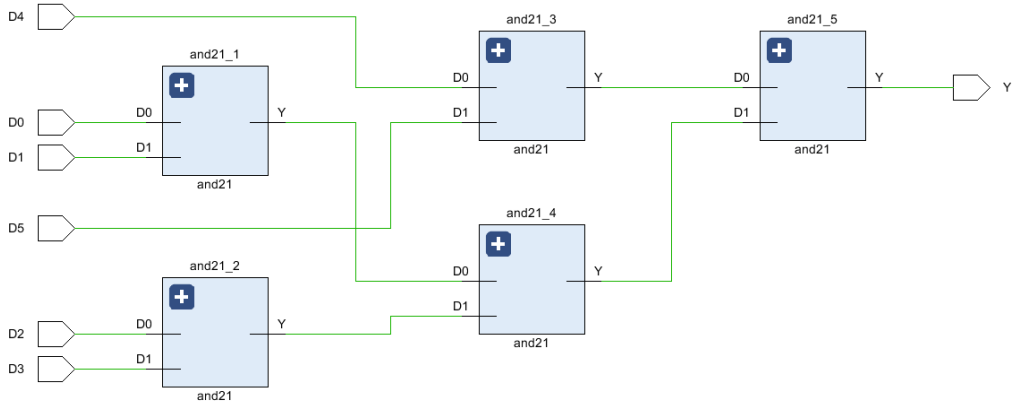
通过设置不同输入变量的并行翻转构成多种情况，使验证结果更为准确。

（2）SIMULATION：



只有当六个输入都为 1 时，输出 Y 为 1，其余情况均为 0。符合电路功能预期。

（3）RTL ANALYSIS:



电路图为 5 个二输入与门通过层级相连，最终构成 6 输入与门的结构。

4.2.3 encoder83 的电路设计

（1）代码分析：

Design:

```
module encoder83_case(I, Y);
    input I;
    output Y;
    wire[7:0] I; //八位二进制输入
    reg[3:1] Y;  //三位二进制输出

    always @(I) begin   //读取到 I 信号时执行
        case(I)
            8'b0000_0001 : Y = 3'b000;
            8'b0000_0010 : Y = 3'b001;
            8'b0000_0100 : Y = 3'b010;
```

```
            8'b0000_1000 : Y = 3'b011;
            8'b0001_0000 : Y = 3'b100;
            8'b0010_0000 : Y = 3'b101;
            8'b0100_0000 : Y = 3'b110;
            8'b1000_0000 : Y = 3'b111;
        endcase
    end
endmodule
```

将输入的独热码 I 编码为对应的 3 位二进制值 Y，使用 case 实现组合逻辑映射，将 I 的对应位转换为二进制数输出。

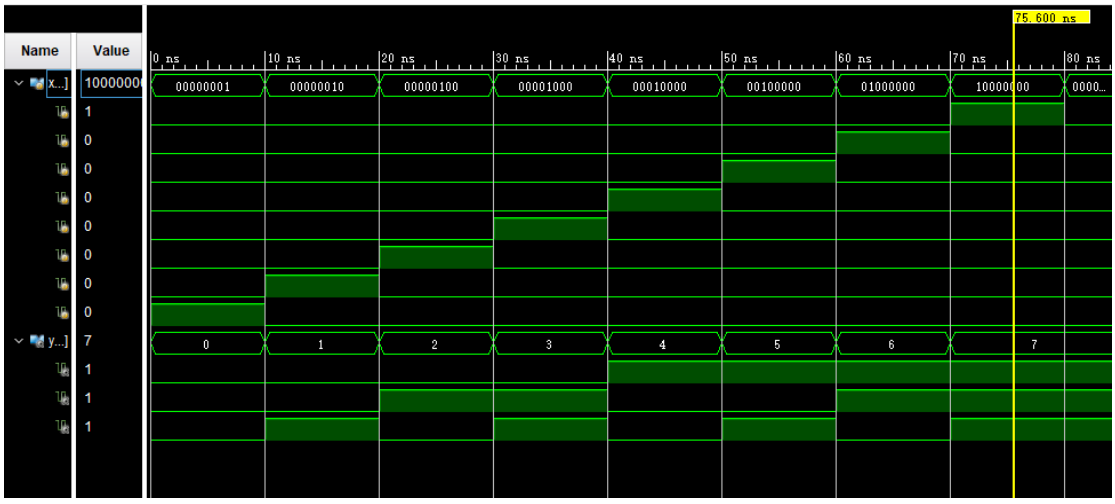
Simulation：

```
module sim_encoder83;
    reg[7:0] x;
    wire[2:0] y;

    encoder83_case test(.I(x),.Y(y)); //实例化
    initial
    begin
        x = 1;
        repeat (7) #10 x = x * 2;    //每 10 单位，x 左移一位，重复七次
        #10 x = 0;                   //将 x 置为 0
        repeat (130) #5 x=x+1;       //每 5 单位给 x 加 1，重复 130 次
    end
endmodule
```

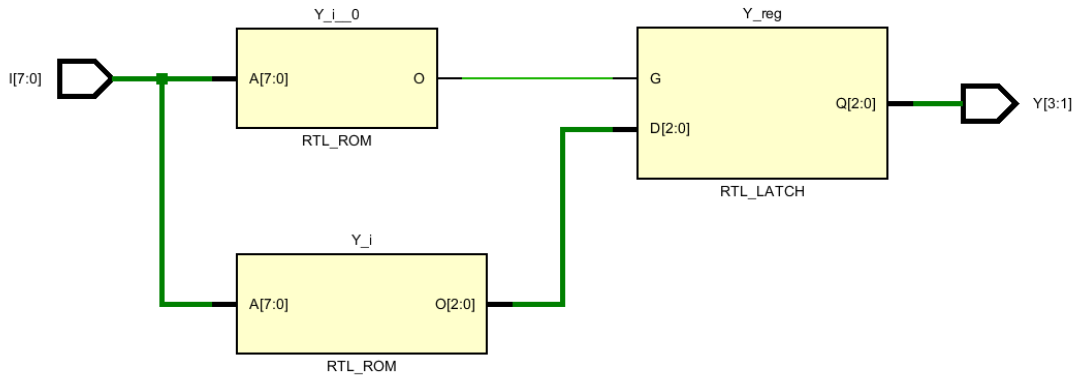
先验证电路功能，再设计其他情况（如 I 中有多个 1 时），观察电路输出。

（2）SIMULATION：



从图中可以看出，I 转换为对应的二进制码，从而实现编码功能。在遇到非法状态的 I 时，输出会延续上一状态的值，直到有新的合法 I 输入。

（3）RTL ANALYSIS:



4.2.4 decoder38 的电路设计

（1）代码分析：

Design:

```
module Decoder38(A, Y);
    input [2:0] A;
    output reg [7:0] Y;
    integer i;

    always @(A) begin
        Y = 8'b1111_1111;
        for(i = 0; i<=7; i = i + 1)
            if (A == i)
                Y[i] = 0;
            else
                Y[i] = 1;
    end
endmodule
```

使用 for 循环将输入三位二进制数 A 解码为独热码输出 Y，比 case 语句更加高效。能够实现将输入值对应位在 Y 中置 0，其余位为 1，如 101→11011111。

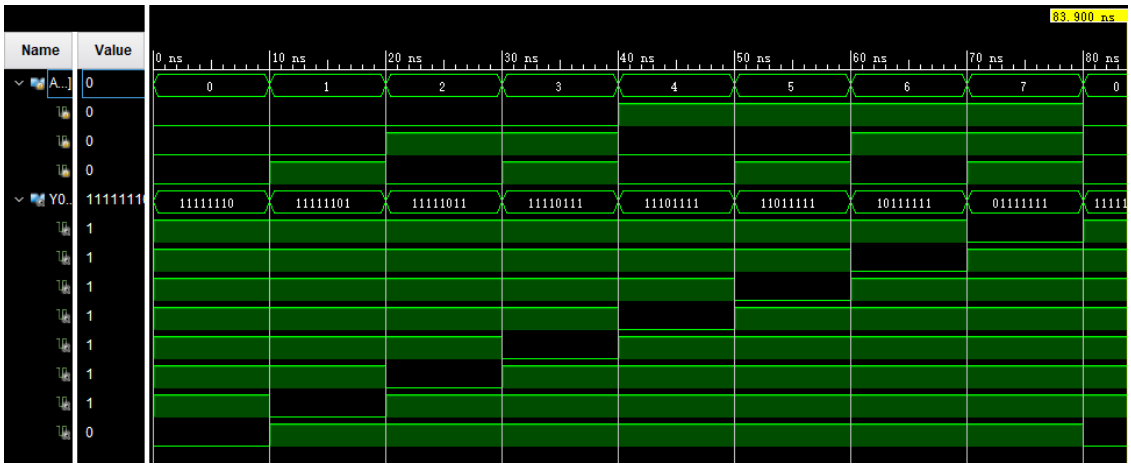
Simulation:

```
module sim_Decoder38;
    reg [3:1] A;
    wire [7:0] YO;
    Decoder38 test(.A(A), .Y(YO));
    initial begin
        A = 3'b000;
        repeat (10) #10 A = A + 1; //每 10 单位给输入加 1，重复 10 次
```

```
    end
endmodule
```

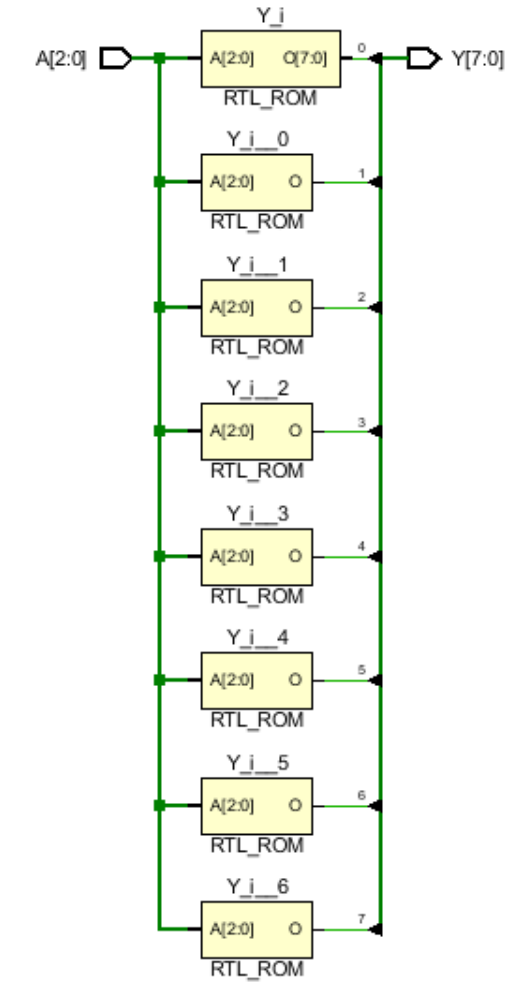
测试从 000 到 111 的输入对应输出。

（2）SIMULATION：



观察波形，可以看出电路成功将 A 转换为对应的 Y，即输入 A 的对应位为 0，符合预期。

（3）RTL ANALYSIS:



4.2.5 触发器的电路设计

（1）代码分析：

Design:

```
module D_ff(Q, QN, D, EN, RST, CLK);
    output reg Q, QN;
    input D;
    input EN, RST, CLK;

    //posedge 为时钟信号上升沿敏感，negedge 为下降沿敏感
    always @(posedge CLK) begin
        if (RST) begin
            Q <= 1'b0;
            QN <= 1'b1;
            end
        else if (EN) begin
            Q <= D; QN <= ~D;
            end
        end
endmodule
```

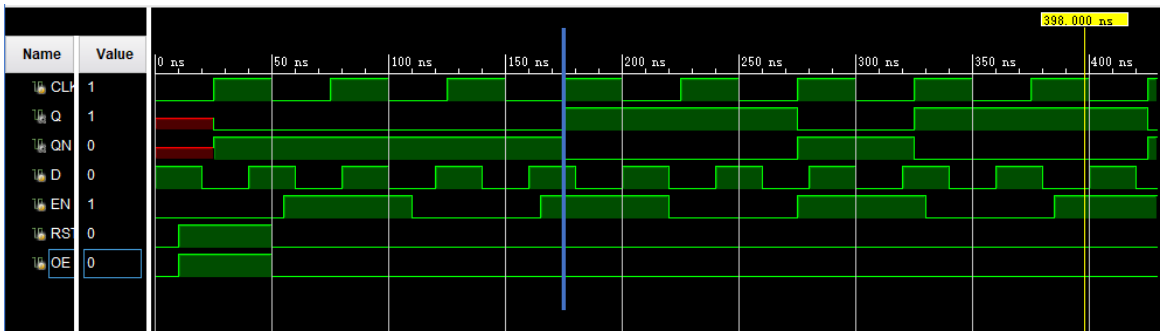
在检测到始终上升沿时，若 RST 为高有效，则进行初始化，将 Q 置为 0，QN（Q 的反）置为 1；若 RST 为低有效且 EN 为高有效，则将 D 传入 Q 输出，D 反传入 QN 输出，实现 D 触发器功能。

Simulation：

```
module sim_D_ff;
    reg CLK;
    wire Q, QN;
    reg D, EN, RST, OE;
    D_ff D_ff_1(Q, QN, D, EN, RST, CLK);
    initial fork
        D = 1; EN = 0; RST = 0; CLK = 0; OE = 0;
        #10 begin RST = 1; OE = 1; end
        #50 begin RST = 0; OE = 0; end
        forever #55 EN = ~EN;
        forever #20 D = ~D;
        forever #25 CLK = ~CLK;
    join
endmodule
```

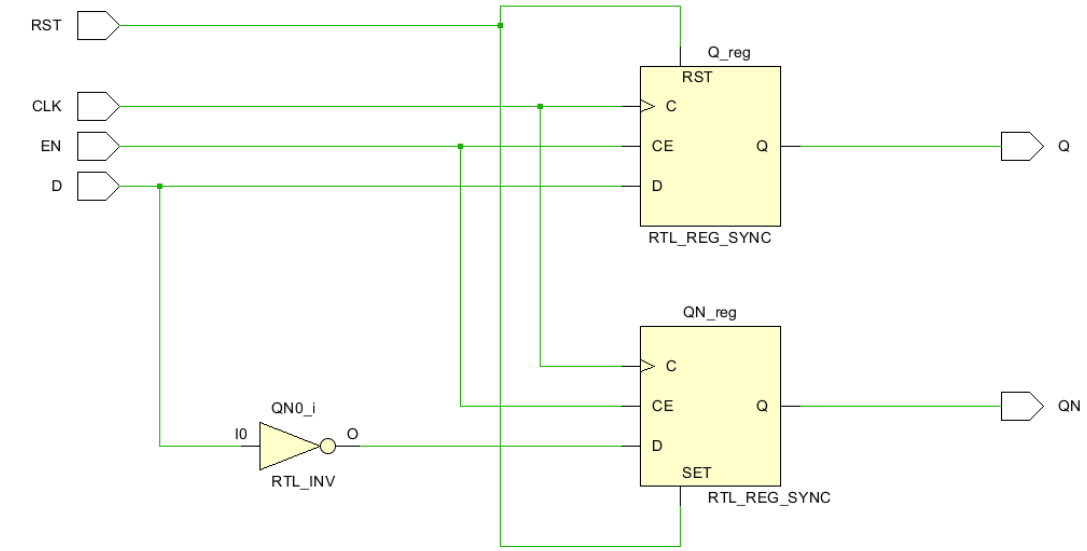
每隔不同时间翻转时钟信号、使能信号和 D，构建多种情况，从而能够更好地观察输出 Q 与 QN 的变化。

（2）SIMULATION：



在蓝线处，时钟处于上升沿，此时 D 和使能信号都为高电平 1，Q=D=1 发生翻转，QN 翻转为 0。此前，没有符合时钟上升沿时 D 与使能都为高电平的情况，因此 Q 一直为初始值 0.

（3）RTL ANALYSIS:



QN 为 Q 的反，因此将 D 取反后接入 QN，其余均相同。

4.2.6 寄存器的电路设计

（1）代码分析：

Design:

```
module register(Q, D, OE, CLK);
    parameter N = 8;
    output reg [N-1:0] Q;
    input [N:1] D;
    input OE, CLK;

    always @(posedge CLK or posedge OE)
        if (OE) Q <= 8'bzzzz_zzzz;
        else Q <= D;
```

```
endmodule
```

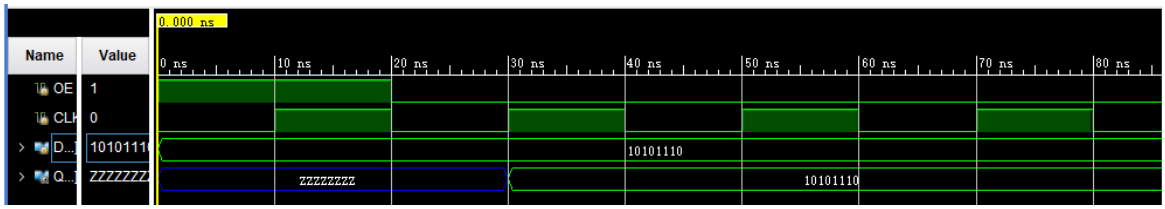
寄存器模块接受 8 位数据 D，在 CLK 上升沿锁存至输出 Q。当 OE 为高时，输出变为高阻态 Z，模拟三态输出功能。

Simulation：

```
module sim_register;
    reg OE, CLK;
    reg [7:0] D;
    wire [7:0] Q;
    register test(.OE(OE), .CLK(CLK), .D(D), .Q(Q));
    initial fork
        OE = 1;
        CLK = 0;
        D = 8'b1010_1110;
        #20 OE = ~OE;
    join
    always #10 CLK = ~CLK;
endmodule
```

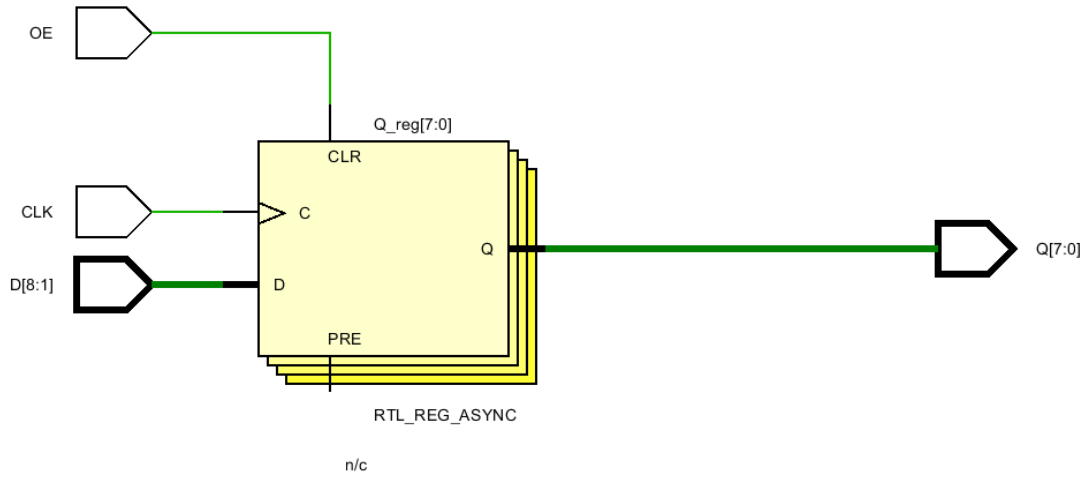
初始 OE=1，寄存器输出为高阻态；在 OE 变化之后观察是否能正确锁存并输出 D；设置时钟周期为 20 时间单位，通过上升沿触发；可观测高阻态转移为有效输出的过程，验证 OE 作用。

（2）SIMULATION：



初始 Q 为 Z，当时钟信号上升沿且 OE 为 0 时，输出 Q 为 D 的值。

（3）RTL ANALYSIS:



4.2.7* 寄存器的电路设计 2

（要求：用结构化描述方式，调用触发器模块实现）

（1）代码分析：

Design:

```
module regD(Q, D, EN, RST, CLK);
    output [7:0] Q;
    input [7:0] D;
    input EN, RST, CLK;
    wire [7:0] QN;

    generate
        genvar i;
        for (i=0; i<8; i=i+1) begin : reg_loop
            D_ff                                                    dff_inst
(.Q(Q[i]), .QN(QN[i]), .D(D[i]), .EN(EN), .RST(RST), .CLK(CLK));
        end
    endgenerate
endmodule
```

调用 8 个 D 触发器，即可实现八位寄存器功能。

Simulation:

```
module sim_regD;
    reg [7:0] D;
    reg EN, RST, CLK, OE;
    wire [7:0] Q;

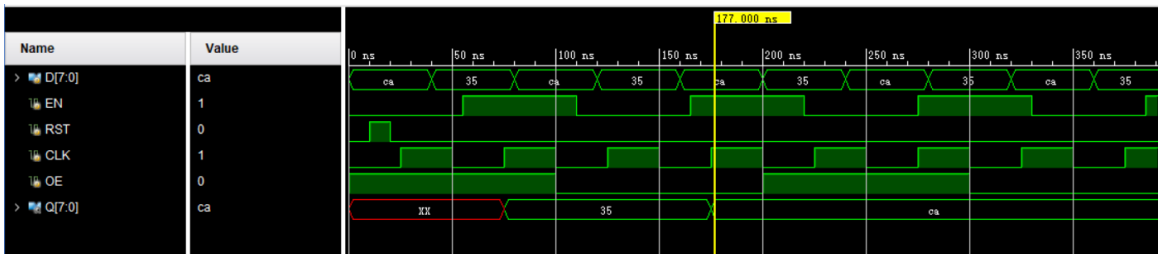
    regD r(Q, D, EN, RST, CLK);

    initial fork
        D = 8'b1100_1010;
        EN = 0;
        RST = 0;
        CLK = 0;
        #10 RST = 1;
        #20 RST = 0;

        forever #55 EN = ~EN;
        forever #40 D = ~D;
        forever #25 CLK = ~CLK;
        forever #100 OE = ~OE;
    join
endmodule
```

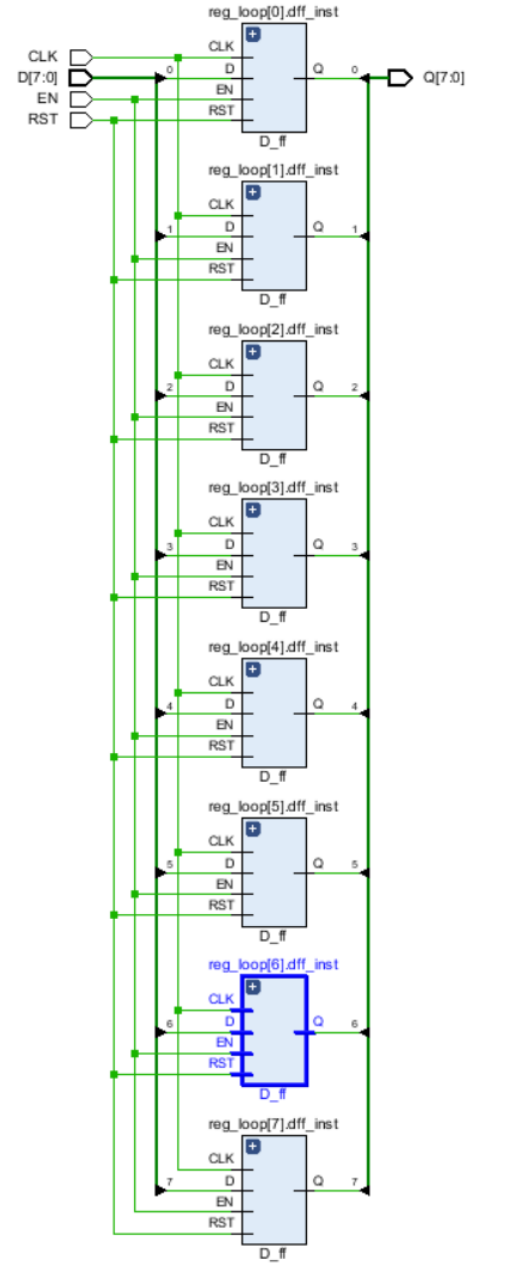
设置各变量每隔不同时间进行翻转，构造多种情况，验证是否达到预期功能。

(2)SIMULATION:



当时钟处于上升沿且 EN 为 1 时（黄线位置），将 D 的值传递给 Q，可见寄存器功能正常实现。

(3)RTL ANALYSIS:



八个 D 触发器级联

五、调试和心得体会

记录实验过程中遇到的问题和调试解决过程，心得体会等。

1、遇到的问题及解决办法：

（1）测试程序报错：不小心拼错代码（如 begin 拼成 beging）。Vivado 感觉很难检查 bug 位置，因此在写代码时一定要仔细认真，尽量不要出错。

（2）在完成调用 D 触发器实现寄存器功能时：

测试结果与预期功能不符，Q 始终为 0：设计文件确认无误后检查 sim 文件，发现问题设计，于是对各变量的翻转周期与初始化情况进行修改，最终得到了与预期相符的波形。

2、实验心得体会：

（1）本次实验帮助我初步认识 Vivado 工具与 Verilog 语言，让我能够理解并运用一些基础的 Verilog 语法，并加深了我对译码器、编码器、D 触发器、寄存器的认识。

（2）在实验中，我意识到模块设计中清晰的端口定义、规范的命名习惯和注释说明对于调试至关重要，并且注意到应该尽量避免小错误，以减少 debug 所需的时间与精力。

（3）从代码编写、仿真、综合、实现到硬件下载的全过程让我熟悉了 Vivado 的开发流程，初步掌握了 FPGA 工程开发基本流程。

（4）学会如何分析 RTL 图结构来验证电路连接是否正确，如何根据仿真波形判断逻辑是否满足预期，为今后更复杂的电路调试打下基础。